



MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR
UNIVERSIDAD SIMÓN BOLÍVAR
DEPARTAMENTO DE COMPUTACIÓN Y CIENCIAS DE LA INFORMACIÓN
TALLER DE BASES DE DATOS I - CI-3391
SARTENEJAS, DISTRITO CAPITAL

Informe Técnico — Corte 3

Docente: Ing. Juan Andres Cuevas

Estudiantes: Cristina Puyosa 23-10395

Alejandro Villamizar 22-10439

Carlos Bisogno 22-10041

Sartenejas, junio 2026

Informe Técnico — Corte 3

Entregable 3A: Funciones Analíticas

get_camera_traffic(camara_id UUID, fecha_inicio DATE, fecha_fin DATE)

Esta función permite obtener el patrón de tráfico de detecciones de una cámara específica, agrupado por hora del día, dentro de un rango de fechas.

```
create or replace function get_camera_traffic(camara_id UUID, fecha_inicio DATE, fecha_fin
DATE)
returns table(total bigINT, hora numeric) AS $$
BEGIN
    return query
    select count(e.EID) as total, extract(hour from e.eTime) as hora
    from "event" as e
    where e.CID = camara_id and e.eTime::date between fecha_inicio and fecha_fin
    group by hora
    order by hora asc;
end;
$$ language plpgsql;
```

La firma de la función, `create or replace function ... returns table(total bigint, hora numeric)`, declara que esta no retorna un valor escalar sino un conjunto de filas con dos columnas (el conteo total de detecciones (`total`) y la hora del día a la que corresponde (`hora`)). Dentro del cuerpo, la instrucción `return query` ejecuta la consulta interna y vierte cada fila resultante directamente como salida de la función, sin necesidad de iterar manualmente con `RETURN NEXT`.

En el `SELECT`, `count(e.EID) as total` cuenta las detecciones agrupadas; se usa `count(e.EID)` en lugar de `count(*)` para contar explícitamente sobre la llave primaria del evento, dejando clara la intención de "número de eventos" y no de "filas resultantes de un producto cartesiano", algo relevante si en el futuro se agregan `JOINS` a esta consulta. Por su parte, `extract(hour from e.eTime) as hora` extrae únicamente la componente de hora (0 a 23) del timestamp del evento, colapsando la dimensión de fecha.

El filtro `e.eTime::date between fecha_inicio and fecha_fin` aplica un cast explícito de `timestampt` a `date` para poder comparar contra el rango de fechas recibido como parámetro, ya que `BETWEEN` requiere operandos de tipos compatibles. Finalmente, `group by hora` agrupa las filas por la hora extraída, necesario por el uso de `count`, y `order by hora asc` ordena el resultado de forma ascendente para presentar un patrón horario legible, de 0h a 23h.

Justificación de diseño

El enunciado pide el conteo "por hora del día" para un rango de fechas. Existían dos interpretaciones posibles: (a) agrupar solo por hora (0–23), sumando las detecciones de todos los días del rango en cada hora, o (b) agrupar por combinación de fecha y hora, generando una fila distinta por cada día. Se optó por la interpretación (a), pues la frase "hora del día" describe un patrón horario recurrente (por ejemplo, "a las 8am esta cámara detecta en promedio X objetos"), y no una bitácora cronológica detallada. Esta interpretación es también consistente con el propósito analítico de la función que

identificar franjas horarias de mayor actividad para una cámara, útil por ejemplo para dimensionar personal de seguridad por turno.

get_zone_summary(tipo_zona VARCHAR)

Para cada ubicación de un tipo de zona dado, retorna el número de cámaras activas, el total de eventos detectados y el total de alertas críticas asociadas.

```
create or replace function get_zone_summary(tipo_zona VARCHAR)
returns table(nombre varchar, num_camaras bigint, num_eventos bigint, num_alertas_criticas
bigint) as $$
begin
    return query
    select l.name,
           count(distinct c.CID) filter (where c.state = 'activa'),
           count(e.EID),
           count(a.AID) filter (where a.severity = 'critica')
    from "location" as l
    left join "camera" as c on c.UID = l.UID
    left join "event" as e on e.CID = c.CID
    left join alert as a on a.EID = e.EID
    where l.zone_type = tipo_zona
    group by l.UID;
end;
$$ language plpgsql;
```

La cadena de joins parte de left join "camera" as c on c.UID = l.UID, donde se utiliza LEFT JOIN en lugar de INNER JOIN para garantizar que toda ubicación del tipo de zona solicitado aparezca en el resultado, incluso si no tiene cámaras asociadas. Con un INNER JOIN, una ubicación sin cámaras desaparecería silenciosamente del resultado, contradiciendo el enunciado, que pide el resumen "para cada ubicación del tipo indicado". Los siguientes joins, left join "event" y left join alert, siguen el mismo razonamiento encadenado: una cámara sin eventos, o un evento sin alertas, no deben eliminar filas del resultado, sino simplemente dejar esas métricas en cero.

En el SELECT, count(distinct c.CID) filter (where c.state = 'activa') combina dos mecanismos. La cláusula FILTER aplica una condición solo a esa agregación puntual, a diferencia de un WHERE que afectaría toda la consulta, mientras que DISTINCT dentro del COUNT es necesario porque, tras los joins con event y alert, una misma cámara puede repetirse múltiples veces en las filas intermedias, sin DISTINCT se sobrecontaría el número de cámaras. Las columnas count(e.EID) y count(a.AID) filter (where a.severity = 'critica') siguen el mismo patrón de conteo condicional para eventos totales y alertas críticas respectivamente.

Por último, group by l.UID agrupa por la llave primaria de location y no por l.name, para evitar que dos ubicaciones con el mismo nombre se fusionen incorrectamente en un solo grupo. PostgreSQL permite proyectar l.name en el SELECT sin incluirlo explícitamente en el GROUP BY, dado que UID determina funcionalmente el resto de las columnas de esa tabla.

Justificación de diseño

El uso de LEFT JOIN encadenado junto con FILTER permite resolver los tres agregados en una sola consulta, evitando subconsultas correlacionadas repetidas (una por cada métrica) que serían menos legibles y más costosas de ejecutar.

find_similar_objects(ref_id UUID, threshold FLOAT, max_results INT)

Dado un objeto de referencia, encuentra los objetos del mismo tipo cuyo embedding visual tenga una distancia coseno menor o igual a un umbral, ordenados por similitud.

```
create or replace function find_similar_objects(ref_id INT, threshold FLOAT, max_results
INT)
returns table(object_ID UUID, object_type varchar(10), distancia_coseno float8, camera_name
varchar(20), tiempo timestamptz) as $$
begin
    return query
        WITH reference_vector as (
            select em.embedding_vec as v
            from "object" as o
            join embedding as em on o.OID = em.OID
            where ref_id = o.OID
        )
        select
            o.OID as object_ID,
            o.object_type,
            (em.embedding_vec <=> rv.v) as distancia_coseno,
            c.name as camera_name,
            e.eTime as tiempo
        from "object" as o
        cross join reference_vector as rv
        join embedding as em on o.OID = em.OID
        join "event" as e on o.EID = e.EID
        join camera as c on e.CID = c.CID
        where (em.embedding_vec <=> rv.v) <= threshold
            and o.OID != ref_id
            and o.object_type = (select object_type from "object" where OID = ref_id)
        order by distancia_coseno asc
        limit max_results;
end;
$$ language plpgsql;
```

La consulta comienza con un CTE, WITH reference_vector as (...), que aísla la obtención del embedding del objeto de referencia mediante un join entre object y embedding filtrado por ref_id. Aislar este cálculo en un CTE separa claramente "el vector con el que se compara" del resto de la lógica, mejorando la legibilidad. Como este CTE produce exactamente una fila, se combina con cada fila candidata de object mediante cross join reference_vector as rv, de forma que rv.v quede disponible como columna en cada comparación de distancia posterior.

En el cuerpo principal, join embedding as em on o.OID = em.OID recupera el embedding de cada objeto candidato que se está evaluando como posible resultado (a diferencia del join equivalente dentro del CTE, que trae el embedding del objeto de referencia) y la expresión (em.embedding_vec <=> rv.v) as distancia_coseno calcula la distancia coseno entre ambos vectores usando el operador <=> de la extensión pgvector, donde valores cercanos a 0 indican mayor similitud. Los joins subsiguientes con "event" y camera navegan la cadena de llaves foráneas object -> event -> camera para obtener el nombre de la cámara y el timestamp del evento donde fue detectado cada objeto candidato, tal como exige la salida especificada en el enunciado.

El WHERE combina tres condiciones. La primera, (em.embedding_vec <=> rv.v) <= threshold, filtra los objetos cuya distancia al vector de referencia no exceda el umbral, usando <= en lugar de <

porque el enunciado especifica "distancia coseno menor o igual al umbral". La segunda, `o.OID != ref_id`, excluye explícitamente al propio objeto de referencia del resultado. La tercera, `o.object_type = (select object_type from "object" where OID = ref_id)`, es una subconsulta escalar que obtiene el tipo del objeto de referencia y filtra los candidatos para que sean del mismo tipo, cumpliendo el requisito de comparar únicamente "objetos del mismo tipo". Finalmente, `order by distancia_coseno asc limit max_results` ordena los candidatos de más a menos similares antes de truncar al número máximo de resultados solicitado, el `ORDER BY` es indispensable antes del `LIMIT`, porque sin él PostgreSQL no garantiza qué subconjunto de filas que cumplen el filtro se retorna.

Justificación de diseño

Se utilizó un CTE en lugar de una subconsulta repetida para el vector de referencia, evitando recalcularlo en cada fila. Se prefirió `pgvector` con índice `hnsw`, declarado en el esquema sobre `embedding.embedding_vec`, para búsquedas de similitud eficientes en espacios de alta dimensionalidad, en lugar de calcular distancias manualmente en Python. Delegar esta operación a la base de datos aprovecha la indexación aproximada de vecinos más cercanos que ofrece la extensión.

Entregable 3B: Triggers

Tabla `cameraAudit` (diseño propio del equipo)

```
create table cameraAudit(  
    AUDID UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
    CID UUID not null,  
    constraint fk_cameraAudit_camera foreign key (CID) references camera(CID),  
    changeTime timestampz not null DEFAULT current_timestamp,  
    oldState varchar(20) not null check (oldState in ('activa', 'en_mantenimiento',  
'inactiva'))  
);
```

Justificación de diseño

`AUDID` se definió como llave primaria propia, independiente de `CID`, porque una misma cámara puede cambiar de estado múltiples veces a lo largo del tiempo, y la tabla de auditoría necesita permitir múltiples filas para una misma cámara, además usar `CID` como PK habría impedido registrar más de un cambio de estado por cámara. `CID` se mantiene como llave foránea `NOT NULL` hacia `camera(CID)`, ya que todo registro de auditoría debe estar vinculado a una cámara existente y no tiene sentido un registro huérfano.

La columna `changeTime` incluye `DEFAULT current_timestamp` para automatizar el registro de la fecha del cambio sin requerir que el trigger la calcule explícitamente, aunque en la implementación se pasa `NOW()` de forma explícita por claridad. `oldState` conserva el mismo `CHECK` de valores permitidos que `camera.state`, lo que mantiene consistencia de dominio entre ambas tablas y evita que se registre un estado anterior inválido que nunca pudo haber existido en `camera`. Finalmente, se optó por no incluir una columna `newState`, dado que el enunciado únicamente solicita el id de la cámara, la fecha del cambio y el estado anterior, el estado nuevo, siempre 'inactiva' por la condición del propio trigger, sería redundante de

almacenar.

trg_alerta_zona_peatonal

Genera automáticamente una alerta de severidad alta cuando se detecta un vehículo en una cámara ubicada en una zona peatonal restringida.

```
CREATE OR REPLACE FUNCTION verif_alerta_peatonal()
RETURNS TRIGGER AS
$$
DECLARE
    e_zone VARCHAR(50);
    e_cam camera%ROWTYPE;
    e_ev "event"%ROWTYPE;
BEGIN
    SELECT * INTO e_ev FROM "event" WHERE EID = NEW.EID;
    SELECT * INTO e_cam FROM camera WHERE CID = e_ev.CID;
    SELECT zone_type INTO e_zone FROM "location" WHERE e_cam.UID = UID;

    IF e_zone = 'peatonal_restringida' AND NEW.object_type = 'vehicle' THEN
        INSERT INTO alert(AID, EID, aTime, severity, "state", description)
        VALUES(
            gen_random_uuid(), NEW.EID, NOW(), 'alta', 'pendiente',
            'El evento corresponde a una cámara en zona peatonal_restringida y el objeto
detectado es un VEHICULO'
        );
    END IF;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

CREATE TRIGGER trg_alerta_zona_peatonal
AFTER INSERT ON "object"
FOR EACH ROW
EXECUTE FUNCTION verif_alerta_peatonal();
```

El bloque DECLARE define tres variables auxiliares: e_zone, de tipo VARCHAR, y e_cam y e_ev, declaradas con %ROWTYPE. Esta última notación permite declarar una variable cuya estructura coincide exactamente con la de una fila de la tabla indicada, sin tener que enumerar cada columna manualmente. A partir de ahí, la función recorre en tres pasos la cadena de llaves foráneas object -> event -> camera -> location: primero SELECT * INTO e_ev FROM "event" WHERE EID = NEW.EID recupera el evento asociado al objeto recién insertado, donde NEW representa la fila de object que disparó el trigger, luego SELECT * INTO e_cam FROM camera WHERE CID = e_ev.CID recupera la cámara que capturó ese evento, y finalmente SELECT zone_type INTO e_zone FROM "location" WHERE e_cam.UID = UID obtiene el tipo de zona de la ubicación de esa cámara.

La condición IF e_zone = 'peatonal_restringida' AND NEW.object_type = 'vehicle' THEN evalúa simultáneamente ambos requisitos del enunciado, y solo si se cumplen los dos se ejecuta el INSERT INTO alert, que crea el registro de alerta con severidad 'alta', como exige el enunciado, y estado inicial 'pendiente', consistente con el flujo de gestión de alertas definido en el esquema. Al cierre, RETURN NEW es obligatorio en toda función de trigger de fila, pues indica qué versión de la fila continúa el

procesamiento; en un trigger AFTER no tiene efecto sobre los datos, ya que estos ya fueron insertados, pero sigue siendo requerido por la sintaxis del lenguaje.

El enunciado original describe el trigger como disparado sobre la inserción del evento de detección. Sin embargo, en el modelo de datos implementado, `object.EID` es una llave foránea hacia `event.EID`, lo cual implica que el evento debe existir antes de poder insertar el objeto detectado en él. Si el trigger se colocara en `event`, en el momento de su ejecución el objeto detectado, y por tanto su `object_type`, aún no existiría en la base de datos, haciendo imposible evaluar la condición "el objeto detectado es un VEHICULO". Por ello, se decidió disparar el trigger en `AFTER INSERT ON "object"`, momento en el cual tanto el evento como el tipo de objeto ya están disponibles, preservando la intención funcional del requisito.

trg_audit_camara

Registra en `cameraAudit` cada vez que una cámara cambia su estado a 'inactiva'.

```
CREATE OR REPLACE FUNCTION verif_audit_camara()
RETURNS TRIGGER AS
$$
BEGIN
    IF NEW."state" <> OLD."state" and NEW."state" = 'inactiva' THEN
        INSERT INTO cameraAudit(AUDID, CID, changeTime, oldState)
        VALUES(gen_random_uuid(), NEW.CID, NOW(), OLD."state");
    END IF;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_camara
AFTER UPDATE ON "camera"
FOR EACH ROW
EXECUTE FUNCTION verif_audit_camara();
```

A diferencia del trigger anterior, aquí se dispone tanto de `NEW` como de `OLD`, ya que en un trigger disparado por `UPDATE` ambas variables especiales están disponibles simultáneamente. `OLD` representa la fila antes del cambio y `NEW` la fila después. La condición `IF NEW."state" <> OLD."state" AND NEW."state" = 'inactiva' THEN` combina dos verificaciones distintas: la primera asegura que el trigger solo actúe si el campo `state` realmente cambió, de modo que un `UPDATE` que modifique otras columnas de `camera` sin tocar `state` no dispare la auditoría y la segunda restringe la auditoría específicamente a transiciones hacia el estado inactivo, tal como exige el enunciado al decir "si el campo estado cambia a inactiva", y no a cualquier cambio de estado en general.

Cuando ambas condiciones se cumplen, `INSERT INTO cameraAudit(...) VALUES(gen_random_uuid(), NEW.CID, NOW(), OLD."state")` registra el id de la cámara, la fecha del cambio, y crucialmente `OLD."state"`, es decir, el estado que tenía la cámara justo antes del cambio, que es el dato que pide capturar el enunciado.

Justificación de diseño

Se utilizan comillas dobles alrededor de state, en NEW."state" y OLD."state", porque esa columna fue declarada como identificador que requiere delimitación explícita en el esquema, según el DDL de camera, y se mantiene esa misma convención al referenciarla dentro del trigger para preservar consistencia.